

OpenDccCLib

Full NMRA DCC Protocol Stack in C

Runs on any processor. No OS. No heap. Just clean C. Edition 2026-09.

Generated 2026-09-25 from the repository sources by documentation/guides/build_pdfs.py

What is DCC?

DCC (Digital Command Control) is the NMRA standard for controlling model railroad locomotives and accessories. A command station sends addressed speed, function and programming commands over the rails as a modulated square wave; decoders in locomotives and accessories receive and act on them. DCC covers short and long addresses, 128-step speed control, 68 function outputs, CV-based configuration, service-mode programming, and bidirectional communication through RailCom.

What is OpenDccCLib?

The full NMRA DCC protocol stack, written in plain C, for command stations and decoders.

"No dynamic memory. No OS. No external dependencies. If your chip has a C compiler and a timer, it can run OpenDccCLib."

Key Features

- **Zero heap.** Every buffer is a static array sized at compile time from `USER_DEFINED_*` constants.
- **Command station and decoder in one library.** Compile-time role flags select what you need; a booster can be both.
- **Full service mode.** Direct, paged, register and address programming, a mode-detect probe, ACK detection with configurable thresholds and a hardware-verified width window.
- **RailCom.** Command-station cutout timed by a five-state machine, armed at the end bit's last edge; 4/8 decode of Channel 1 and 2 replies, tagged with the packet's address. Decoder-side reply engine with bit-banged 4/8 transmit.

- **Spec-correct repeats.** Every packet builder carries the repeat count the standard requires: two identical packets for a CV write, three for a date, one for a verify.
- **Dependency injection.** Hardware is reached only through function pointers wired at initialization. Port by rewriting one driver file; the protocol code is untouched.
- **Proven on the wire.** Beyond the unit tests, a Saleae hardware-in-the-loop bench checks the shipped firmware against the NMRA standards, down to microsecond timing and per-packet repeat counts.
- **BSD 2-Clause.** Use in open-source or commercial products, no royalties.

Protocol Coverage

Module	Details
Bit encoder	NMRA 58 μ s one-bit and 100 μ s zero-bit halves from a shared 58 μ s timer; single-buffered serialization from the ISR; RailCom cutout armed at the end bit's last edge
Packet builders	Speed 14/28/128, F0–F68, basic and extended accessories with CV access and NOP, consist, binary state, analog, time and date, loco CV access (POM); each with its spec-correct repeat count
Scheduler	Static slots with priority (ESTOP > SPEED > FUNCTION > ACCESSORY > CV > IDLE), duplicate combining by (address, tag), auto-refresh round-robin, 5 ms same-address spacing
Service mode	Direct, paged, register, address primitives plus read/write/verify task orchestrators and mode detection; configurable ACK threshold, window and retries
RailCom cutout	DELAY 26, SETTLING 54, CH1 97, GAP 16, CH2 261 μ s (454 μ s total); user-configurable at compile time and at runtime; cancel and status calls
RailCom receive	Window-gated 250 kbaud UART hooks, standard S-9.3.2 4/8 code words, ACK and NACK, Channel 1 and 2 datagram assembly, address tagging, receive ring
Bit decoder	Edge-timing classification with an 80 μ s threshold, stretched-zero tolerance, 10 ms no-signal recovery
Packet decoder	Preamble $\geq$ 10, byte assembly, XOR check, matching for short, long, consist and accessory addresses, deferred dispatch queue so callbacks never run in an interrupt
CV storage	Function-pointer storage with decoder lock (CV 15/16), factory reset on CV 8, indexed CVs (CV 31/32, 257–512), CV 29 feature mask hook
RailCom transmit	Decoder reply engine: automatic Channel 1 address broadcast, application-filled Channel 2, ACK/BUSY/NACK tokens, accessory SRQ and status
Fail-safe	S-9.2.4 packet time-out from CV 11 in 100 ms units, with enter and exit callbacks

Example Platforms

Two ready-to-run projects ship with the library. Porting to a new platform means rewriting one timer/GPIO driver file and, optionally, a UART driver.

Example	Platform	Toolchain
Command station	TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia
Decoder	TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia

Architecture Highlights

- One `dcc_config.h` include, one `dcc_config_t` struct, one `DccConfig_initialize()` call.
- Every module talks to its neighbours through an interface struct, so each is mocked in isolation and an MCU swap touches only the driver layer.
- A single `DccConfig_run()` per main-loop pass drives the whole engine; every application callback fires there, never from an interrupt.
- A shared 58 μ s timer clocks both the main and the service track; the decoder's edge interrupt does nothing but timestamp.
- Compile-time role selection: command station, decoder, accessory decoder, or any combination.

Test Coverage

Metric	Value, measured at generation (2026-09-24)
Unit-test binaries	29 GoogleTest binaries, one per module
Test cases	1144, all passing, zero warnings
Coverage (gcovr)	95.4 % lines, 97.7 % functions, 87.8 % branches
Compile-gate matrix	Six single-role configurations built and linked on every run
Source	60 files, 17 959 lines in <code>src/dcc/</code>
Hardware-in-the-loop	7 Saleae suites, 393 on-wire checks: S-9.1, S-9.2, S-9.2.1, S-9.2.3, S-9.3.2, scheduler (incl. refresh pacing), plus a bench preflight

Getting Started

1. **Command station.** Import `applications/ti_theia/mspm03507_launchpad/command_station/` into CCS, build, flash, open a 230400-baud terminal and type `POWER ON`. DCC on the track in minutes.
2. **Decoder.** Import `.../decoder/`, build, flash, connect a DCC source. Every decoded command prints as a `RECV` line.
3. **New platform.** Copy an example, rewrite the driver files for your timers and GPIO. The protocol engine runs unchanged.

Documentation

- Quick Start Guide, Command Station and Decoder: step-by-step LaunchPad walkthroughs.
- Developer Guide, Command Station and Decoder: config struct, internals, porting, testing.
- `documentation/ARCHITECTURE.md`: the as-built design. `documentation/compliance/`: per-feature NMRA compliance status with machine-checked test references.
- API reference: Doxygen from the headers.
- github.com/JimKueneman/OpenDccCLib, BSD 2-Clause.

Built for model railroaders, by a model railroader.